

¿Qué hace este programa?

Explicación para principiantes totales

*De una regla para aceptar palabras
a la estructura matemática que la define*

Proyecto final · Matemática Discreta II & Teoría de la Computación

La idea en una sola frase

Escribes una **regla** para aceptar o rechazar palabras; el programa construye la **máquina** que la cumple, observa **qué le hace cada palabra** a esa máquina, comprime las infinitas palabras en un **objeto pequeño**, y te dice **a qué familia matemática pertenece** tu regla: un interruptor, un reloj, un detector de patrones...

Vamos a construir todo desde cero, como una torre de bloques: cada concepto se apoya en el anterior. Sin prisa y sin dar nada por sabido.

1. La portera del club (el autómatá)

Imagina una **máquina portera** en la puerta de un club. Le muestras una palabra (una cadena de letras) y solo sabe hacer dos cosas: decir «**pasas**» (acepta) o «**no pasas**» (rechaza).

La regla que usa para decidir se llama un **lenguaje**. Por ejemplo: «*dejo pasar solo las palabras que terminan en 01*», o «*solo las que tienen un número par de unos*».

Un **autómata** (en el libro, **AFD**) es la portera más simple posible:

- Tiene unos **estados**, como «estados de ánimo». Piensa en focos: en cada momento hay exactamente uno encendido.
- Lee tu palabra **letra por letra, de izquierda a derecha**, sin volver atrás jamás.
- Cada letra la hace **saltar** de un foco a otro, siguiendo flechas fijas.
- Al terminar, mira en qué foco quedó. Si es un foco «bueno» (de aceptación), dice «**pasas**».

Ejemplo: «número par de unos»

Foco **A** = «he visto una cantidad *par* de unos» (el bueno). Foco **B** = «cantidad *impar* de unos».

Cada 1 que lee la hace saltar al otro foco; los ceros no cambian nada. Empieza en A. Si al final sigue en A $\Rightarrow$ par $\Rightarrow$ **pasas**.

Un autómatá es **memoria mínima**: solo recuerda «en qué foco estoy», nada más. Por eso es rapidísimo, pero también limitado.

2. La expresión regular (la forma cómoda de escribir la regla)

Dibujar focos y flechas a mano es tedioso. Una **expresión regular** (o *regex*) es una forma corta de **escribir** la regla con símbolos:

- 01 = «un cero seguido de un uno».
- $a|b$ = «una **a** o una **b**».

- $a^* = \text{«cero o más a seguidas»}$: "", a, aa, ...
- $(0|1)^*01 = \text{«cualquier cosa hecha de ceros y unos que termine en 01»}$.

Tú escribes la regex, y **el programa te construye la portera automáticamente**. Lo hace en tres pasos internos:

1. **Leer la regex** y entenderla como una receta.
2. **Armar una primera portera** a partir de la receta (rápida de construir, pero «desordenada»).
3. **Ordenarla y encogerla**: fusionar los focos que se comportan igual hasta obtener la portera **más pequeña posible**. Esta versión mínima será clave en el paso final.

3. El monoide: la parte estrella ★

Aquí está la idea más bonita del proyecto. La voy a explicar con mucho cuidado.

La pregunta clave

Ya tenemos la portera. Ahora nos hacemos una pregunta *distinta*:

*«¿Qué le hace **cada palabra** a los focos de la portera?»*

Una palabra no solo se acepta o rechaza: una palabra es una **instrucción de reordenamiento** de los focos. Una *coreografía*.

Cada palabra es una coreografía

Volvamos a la portera de «par de unos» (dos focos, A y B). Pregúntate qué hace la palabra 1:

- Si estabas en A, te manda a B. Si estabas en B, te manda a A.

O sea, 1 es la instrucción **«intercambia los dos focos»**. Ahora la palabra 11: de A pasas a B y vuelves a A; de B a B y vuelves a B. La palabra 11 es la instrucción **«déjalo todo igual»**.

Y resulta que 1, 111, 101... todas hacen «intercambia»; mientras que "", 11, 1001... todas hacen «deja igual».

El descubrimiento

Aunque hay **infinitas palabras**, ¡en esta portera solo existen **dos coreografías posibles!** «*deja todo igual*» y «*intercambia los dos focos*».

A la colección de coreografías distintas se le llama el **monoide de transición**, escrito $M(A)$. Es como haber comprimido infinitas palabras en un puñadito de «movimientos esenciales».

¿Cómo lo calcula el programa?

Con mucha paciencia y de forma automática:

1. Empieza con la coreografía «no hacer nada» (la palabra vacía).
2. Le añade **una letra** y anota la coreografía resultante.
3. Si esa coreografía **ya la había visto**, la ignora; si es **nueva**, la guarda.
4. Repite hasta que **no aparecen coreografías nuevas**.

Al terminar tiene la lista completa. Para «par de unos» encuentra 2; para otras reglas encontrará 3, 4, 5...

Combinar coreografías

Lo mágico: hacer una coreografía y luego otra da como resultado **otra coreografía de la misma lista** (por ejemplo, «intercambia» + «intercambia» = «deja igual»). El programa arma una **tabla** —como una tabla de multiplicar— que dice qué coreografía resulta de encadenar cualquier par. Esa tabla es la que se examina en el último paso.

4. La «personalidad» de la regla

Con esa tabla, el programa descubre **de qué tipo matemático** es tu regla, y lo muestra como etiquetas en la página:

- **¿Es un «grupo»?** Un grupo es un conjunto de movimientos donde *todo movimiento se puede deshacer*. «Par de unos» lo es: «intercambia» se deshace con otro «intercambia». Y ese grupo resulta ser el **mismo objeto matemático** que un interruptor de luz o que sumar pares e impares: se llama $\mathbb{Z}/2\mathbb{Z}$.
- Otras reglas dan grupos más ricos: $\mathbb{Z}/3\mathbb{Z}$ (como un reloj de 3 horas), V_4 , S_3 ... El programa reconoce cuál es.
- **¿Es «aperiódico»?** Significa que la regla *no cuenta en ciclos*: si repites una misma palabra muchas veces, la coreografía deja de cambiar (se «estabiliza»), en vez de girar en un ciclo como hace «intercambia». La regla solo detecta patrones («¿apareció 01 al final?»). «Termina en 01» es de este tipo.

Un detalle honesto (y por qué importaba encoger la portera)

Para que ese objeto sea *el correcto*, la portera debe ser la **mínima**. Un teorema llamado **Myhill–Nerode** garantiza que, con la portera mínima, el objeto que sale es **único y verdadero** para esa regla. Por eso el paso de «ordenar y fusionar focos» no era un adorno: era esencial.

5. La Máquina de Turing (la segunda sección de la página)

La portera es sencilla, pero **hay reglas que no puede resolver**. Por ejemplo: «*acepta $a^n b^n$* », es decir, la misma cantidad de a que de b , en ese orden ($aabb$ sí, aab no). No puede, porque **no sabe contar sin límite**: su memoria es fija.

La **Máquina de Turing** es la portera con superpoderes: tiene una **cinta de papel infinita** donde puede **escribir, borrar y moverse** a izquierda y derecha cuantas veces quiera. Con eso sí puede contar y comparar. Su truco típico es **marcar y emparejar**: tacha una a , busca una b y la tacha, vuelve al inicio y repite. En la página ves la cinta moverse paso a paso, con el cabezal (la cabeza lectora) animado.

¿Y el álgebra aquí?

Si el lenguaje que describes es **simple** (regular), el programa construye su portera mínima y hace **exactamente el mismo análisis** de las secciones anteriores.

Si el lenguaje **necesita los superpoderes** (como $a^n b^n$), el programa te dice con honestidad que aquí el objeto ya no es un puñadito finito de coreografías, sino **infinito**, y te explica por qué. No lo esconde: te muestra justo dónde está la frontera.

Resumen en una frase

Escribes una regla $\rightarrow$ el programa construye la máquina que la cumple $\rightarrow$ mira qué le hace cada palabra a la máquina $\rightarrow$ comprime infinitas palabras en un objeto pequeño $\rightarrow$ y te dice a qué familia matemática pertenece tu regla.

Autómatas y Máquinas de Turing por un lado; el puente hacia el álgebra abstracta de Matemática Discreta II por el otro. Eso es todo el programa.